



Flow-based Deep Generative Models

<https://hku-data8018.github.io/>

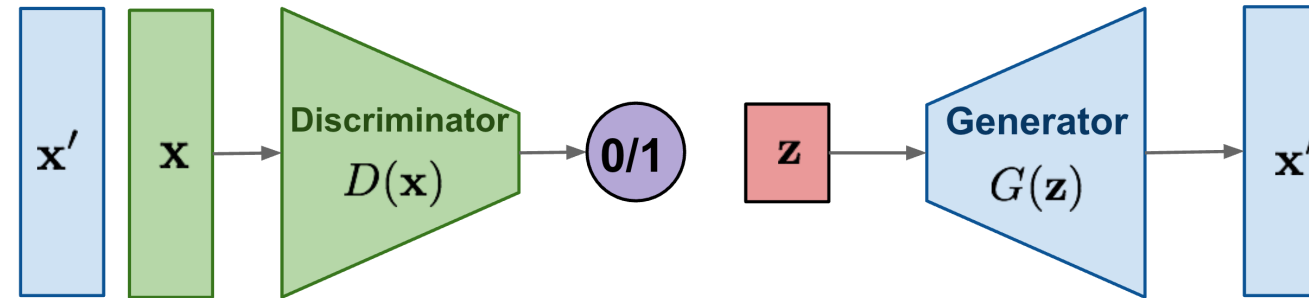
Bo Dai

Spring 2026

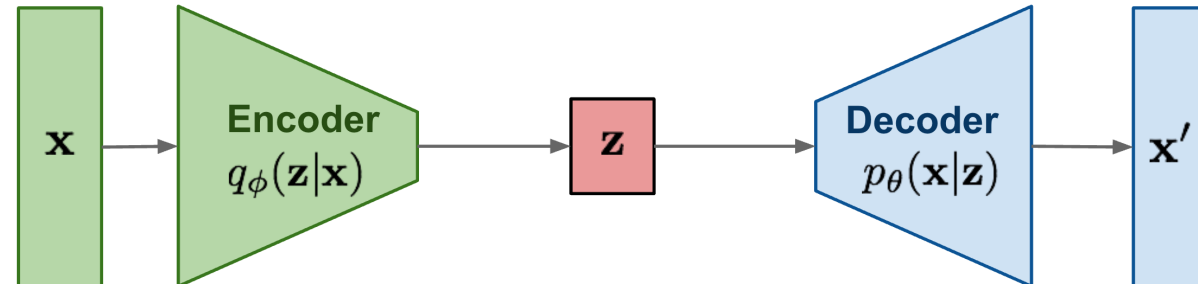


- Slides reference
 - Stanford Fall 2023, CS236 Deep Generative Models, <https://deepgenerativemodels.github.io/>
 - <https://lilianweng.github.io/posts/2018-10-13-flow-models/>
 - Peking University, <https://deep-generative-models.github.io/>

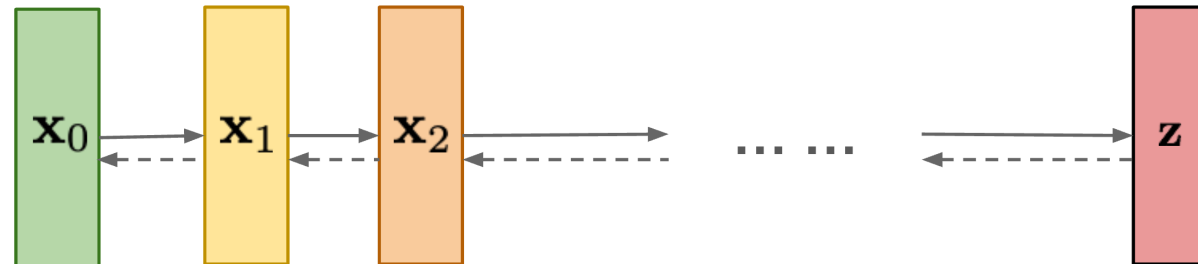
GAN: Adversarial training



VAE: maximize variational lower bound

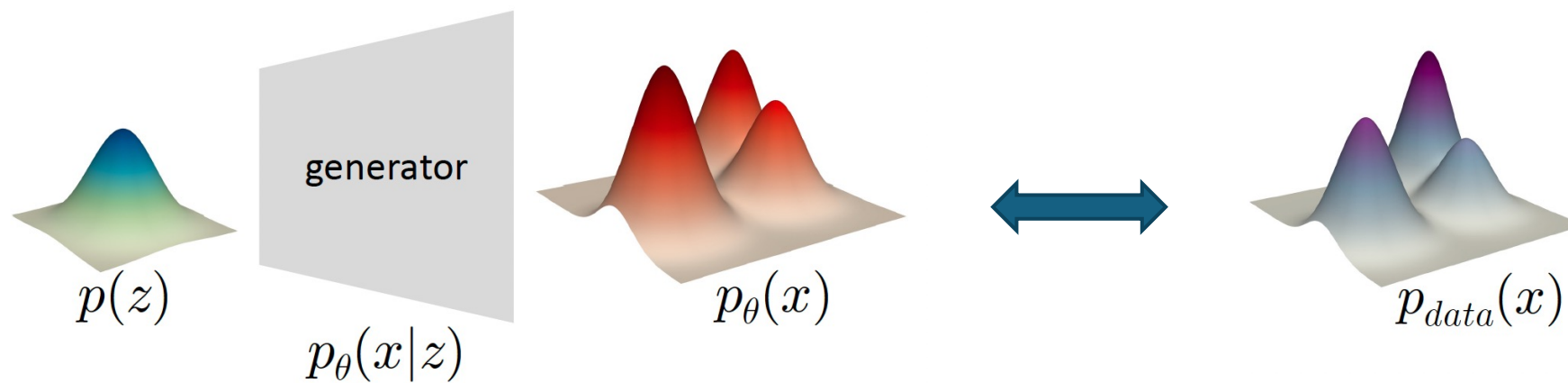


Diffusion models:
Gradually add Gaussian noise and then reverse



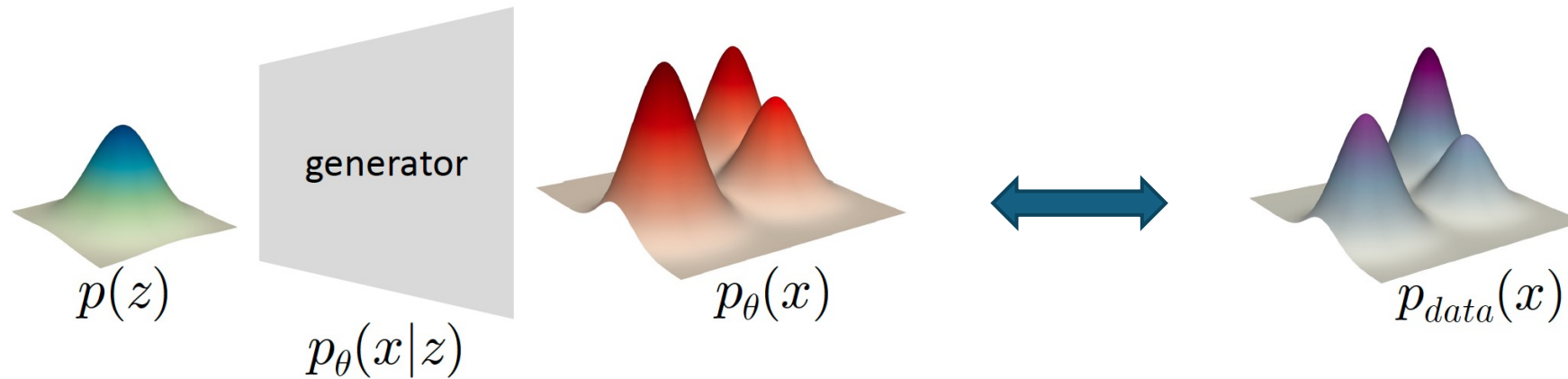
Approximate the true data distribution $p_{\text{data}}(x)$ with learned distribution $p_{\theta}(x)$

$$\mathbb{E}_{x \sim p_{\text{data}}} \log p_{\theta}(x)$$



~~Approximate the true data distribution $p_{data}(x)$ with learned distribution $p_{\theta}(x)$~~
How about compute **the exact likelihood** without approximation?

$$\mathbb{E}_{x \sim p_{data}} \log p_{\theta}(x)$$



Background: Change of Variable Theorem (1D)

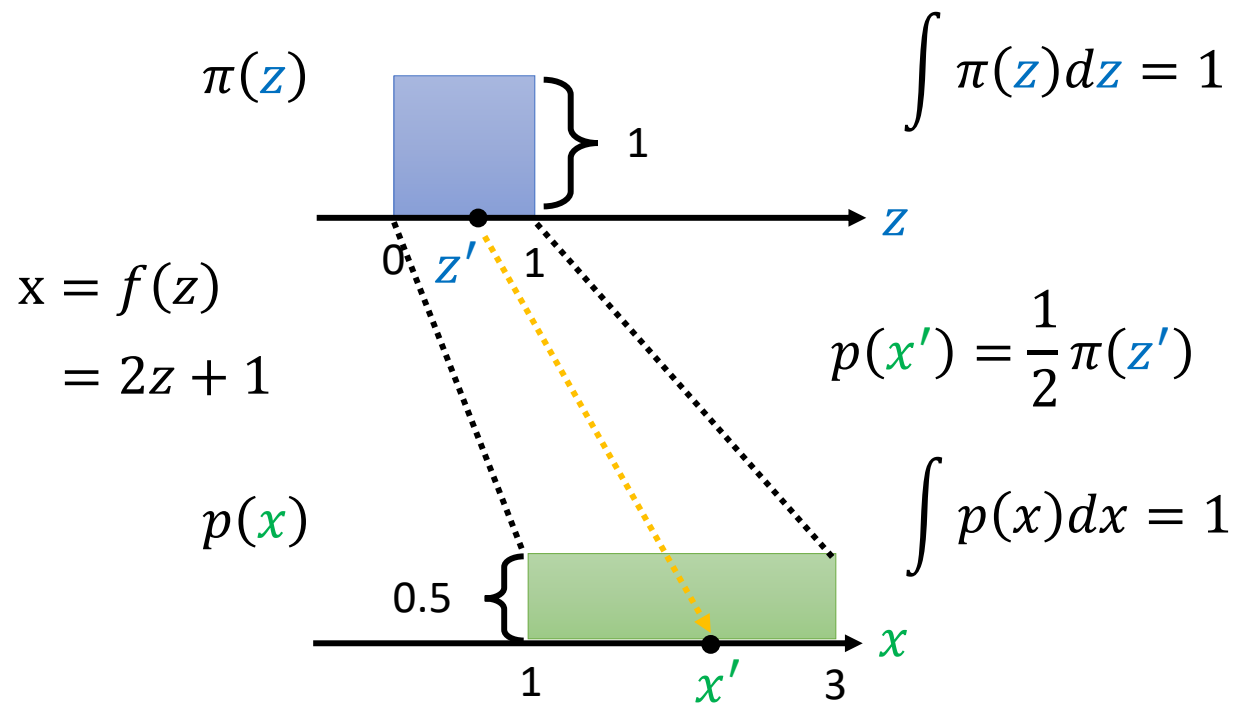
- Let z be a uniform random variable $U[0,1]$ with density π_z . What is $\pi_z(1/2)$?
 - $1, \int_0^1 1 = 1$
- Let $x = f(z) = 2z + 1$ and let p_x be its density. What is $p_x(2)$?
 - $p_x(2) = \pi_z((2 - 1) / 2) = \pi_z(1/2) = 1?$

Background: Change of Variable Theorem (1D)

- Let z be a uniform random variable $U[0,1]$ with density π_z . What is $\pi_z(1/2)$?
 - $1, \int_0^1 1 = 1$
- Let $x = f(z) = 2z + 1$ and let p_x be its density. What is $p_x(2)$?
 - ~~$p_x(2) = \pi_z((2-1)/2) = \pi_z(1/2) = 1?$~~
 - x is uniform in $[1,3]$, so $p_x(x) = 1/2$

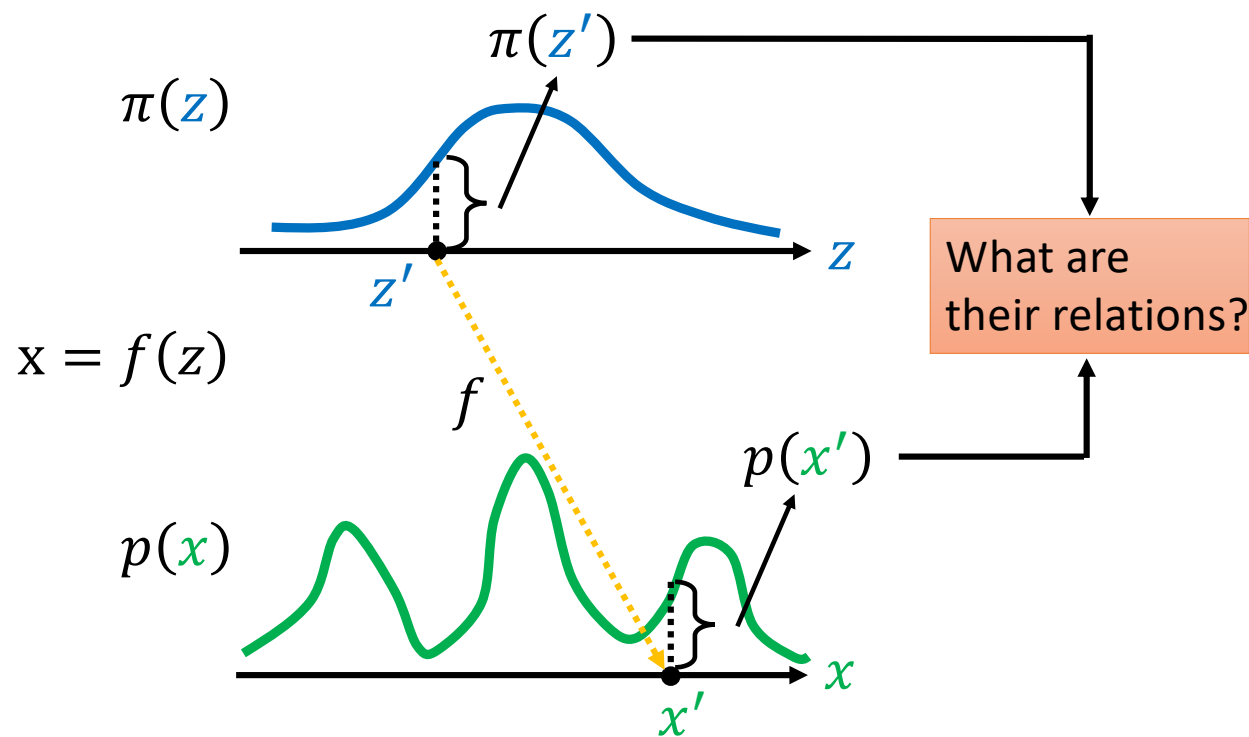
Background: Change of Variable Theorem (1D)

- Let z be a uniform random variable $U[0,1]$ with density π_z . What is $\pi_z(1/2)$?
 - 1, $\int_0^1 1 = 1$
- Let $x = f(z) = 2z + 1$ and let p_x be its density. What is $p_x(2)$?



Background: Change of Variable Theorem (1D)

- Let z be a uniform random variable $U[0,1]$ with density π_z . What is $\pi_z(1/2)$?
 - $1, \int_0^1 1 = 1$
- Let $x = f(z) = 2z + 1$ and let p_x be its density. What is $p_x(2)$?



Background: Change of Variable Theorem (1D)

- When $x = f(z)$ and function f is **invertible** and **differentiable**.
- If f is monotonically increasing, we have

$$Pr(z' \leq z \leq (z' + \Delta z)) = Pr(f(z') \leq f(z) \leq f(z' + \Delta z)) = Pr(x' \leq x \leq (x' + \Delta x))$$

- Similar for monotonically decreasing

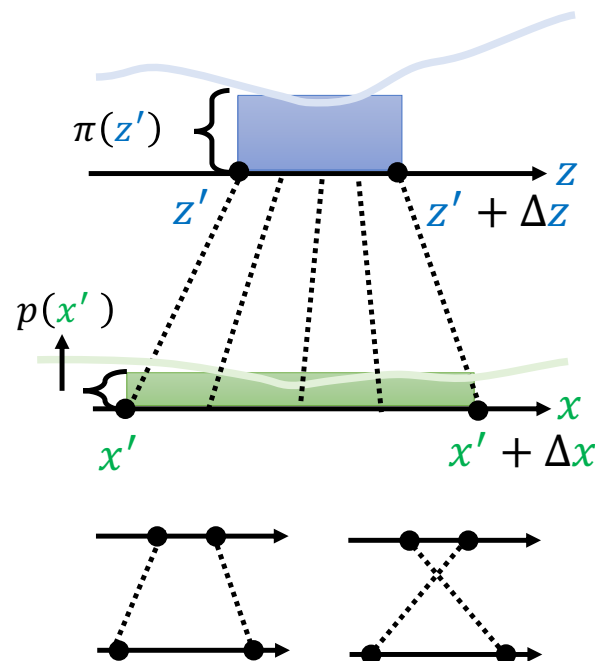
$$\left| \int_{z'}^{z' + \Delta z} \pi(z) dz \right| = \left| \int_{x'}^{x' + \Delta x} p(x) dx \right|$$

- When $\Delta z \rightarrow 0$, we have $p(x') = \pi(z') \left| \frac{\Delta z}{\Delta x} \right|_{x=x'} = \pi(z') \left| \frac{dz}{dx} \right|_{x=x'}$

Background: Change of Variable Theorem (1D)

- When $x = f(z)$ and function f is **invertible** and **differentiable**.
- If f is monotonically increasing, we have

$$Pr(z' \leq z \leq (z' + \Delta z)) = Pr(f(z') \leq f(z) \leq f(z' + \Delta z)) = Pr(x' \leq x \leq (x' + \Delta x))$$



The blue square and the green square should be equal in area

$$p(x')|\Delta x| = \pi(z')|\Delta z|$$

$$p(x') = \pi(z') \left| \frac{dz}{dx} \right|$$

Background: Change of Variable Theorem (1D)

- When $x = f(z)$ and function f is **invertible** and **differentiable**

$$p(x) = \pi(z) \left| \frac{dz}{dx} \right| = \pi(z) \left| \frac{df^{-1}(x)}{dx} \right|$$

- How about multi-dimensional cases?

Background: Change of Variable Theorem

- Jacobian Matrix

$$\begin{aligned} 1) \quad x &= \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \quad z = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} \\ x &= f(z) \quad z = f^{-1}(x) \end{aligned}$$

$$2) \quad \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} z_1 + z_2 \\ 2z_1 \end{bmatrix} = f\left(\begin{bmatrix} z_1 \\ z_2 \end{bmatrix}\right)$$

$$\begin{bmatrix} x_2/2 \\ x_1 - x_2/2 \end{bmatrix} = f^{-1}\left(\begin{bmatrix} x_1 \\ x_2 \end{bmatrix}\right)$$

$$3) \quad J_f = \begin{bmatrix} \overbrace{\partial x_1 / \partial z_1}^{\text{input}} & \partial x_1 / \partial z_2 \\ \partial x_2 / \partial z_1 & \partial x_2 / \partial z_2 \end{bmatrix} \Bigg|_{\text{output}}$$

$$J_{f^{-1}} = \begin{bmatrix} \partial z_1 / \partial x_1 & \partial z_1 / \partial x_2 \\ \partial z_2 / \partial x_1 & \partial z_2 / \partial x_2 \end{bmatrix}$$

4)

$$J_f = \begin{bmatrix} 1 & 1 \\ 2 & 0 \end{bmatrix}$$

$$J_{f^{-1}} = \begin{bmatrix} 0 & 1/2 \\ 1 & -1/2 \end{bmatrix}$$

$$J_f J_{f^{-1}} = I$$

Background: Change of Variable Theorem

- Jacobian Matrix
- Determinant: The determinant of a **square matrix** is a **scalar** that provides information about the matrix

• 2 X 2

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

$$\det(A) = ad - bc$$

$$\det(A) = 1/\det(A^{-1})$$

$$\det(J_f) = 1/\det(J_{f^{-1}})$$

• 3 x 3

$$A = \begin{bmatrix} a_1 & a_2 & a_3 \\ a_4 & a_5 & a_6 \\ a_7 & a_8 & a_9 \end{bmatrix}$$

$$\det(A) =$$

$$a_1 a_5 a_9 + a_2 a_6 a_7 + a_3 a_4 a_8$$

$$- a_3 a_5 a_7 - a_2 a_4 a_9 - a_1 a_6 a_8$$

Background: Change of Variable Theorem

- If the mapping function between $\mathbf{z}$ and $\mathbf{x}$, given by $f: \mathbb{R}^n \rightarrow \mathbb{R}^n$, is differentiable and invertible, such that $\mathbf{x}=f(\mathbf{z})$ and $\mathbf{z} = f^{-1}(\mathbf{x})$, then

$$p(\mathbf{x}) = \pi(\mathbf{z}) \left| \det\left(\frac{\partial f^{-1}(\mathbf{x})}{\partial \mathbf{x}}\right) \right| = \pi(\mathbf{z}) | \det(J_{f^{-1}}) |$$

- $\mathbf{x}$ and $\mathbf{z}$ need to be continuous and have the same dimension
- Since for any invertible matrix A , $\det(A^{-1}) = \det(A)^{-1}$

$$p(\mathbf{x}) = \pi(\mathbf{z}) \left| \frac{1}{\det(J_f)} \right|$$

Normalizing flow models

- **Normalizing**: change of variables gives a normalized density after applying an invertible transformation
- **Flow**: Invertible transformations can be composed with each other

$$\mathbf{z}_m = \mathbf{f}_\theta^m \circ \cdots \circ \mathbf{f}_\theta^1(\mathbf{z}_0) = \mathbf{f}_\theta^m(\mathbf{f}_\theta^{m-1}(\cdots(\mathbf{f}_\theta^1(\mathbf{z}_0)))) \triangleq \mathbf{f}_\theta(\mathbf{z}_0)$$

Normalizing flow models

- **Normalizing**: change of variables gives a normalized density after applying an invertible transformation
- **Flow**: Invertible transformations can be composed with each other

$$\mathbf{z}_m = \mathbf{f}_\theta^m \circ \cdots \circ \mathbf{f}_\theta^1(\mathbf{z}_0) = \mathbf{f}_\theta^m(\mathbf{f}_\theta^{m-1}(\cdots(\mathbf{f}_\theta^1(\mathbf{z}_0)))) \triangleq \mathbf{f}_\theta(\mathbf{z}_0)$$

- Start with a simple distribution for $\mathbf{z}_0$ (e.g. Normal distribution)
- Apply a sequence of M invertible transformations to finally obtain $\mathbf{x} = \mathbf{z}_M$
- By change of variables

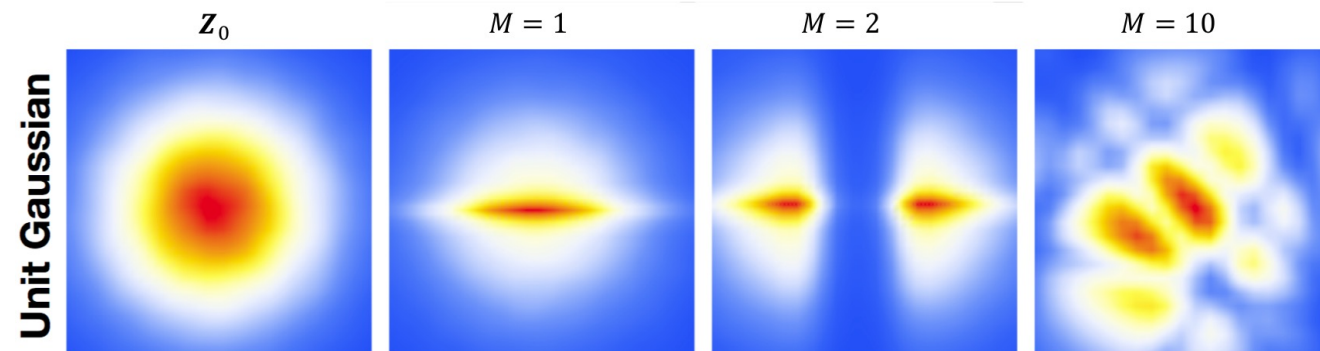
$$p_{\mathbf{x}}(\mathbf{x}; \theta) = p_{\mathbf{z}}(\mathbf{f}_\theta^{-1}(\mathbf{x})) \prod_{m=1}^M \left| \det \left(\frac{\partial (\mathbf{f}_\theta^m)^{-1}(\mathbf{z}_m)}{\partial \mathbf{z}_m} \right) \right|$$

Normalizing flow models

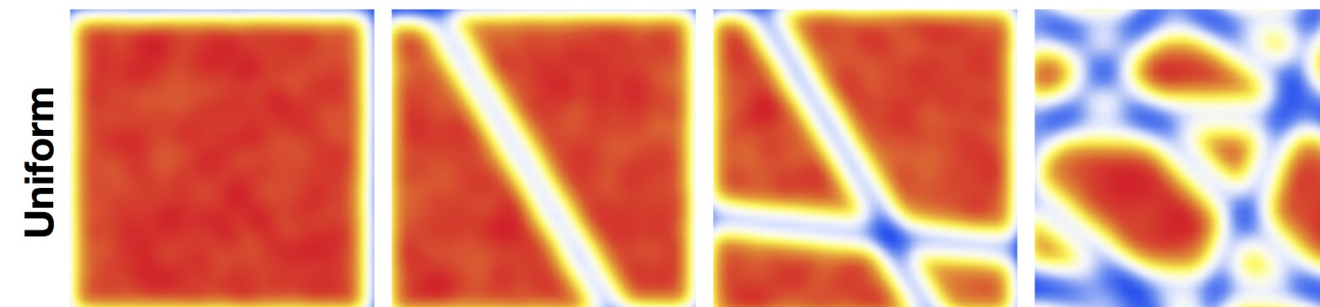
- **Normalizing**: change of variables gives a normalized density after applying an invertible transformation
- **Flow**: Invertible transformations can be composed with each other

$$\mathbf{z}_m = \mathbf{f}_\theta^m \circ \cdots \circ \mathbf{f}_\theta^1(\mathbf{z}_0) = \mathbf{f}_\theta^m(\mathbf{f}_\theta^{m-1}(\cdots(\mathbf{f}_\theta^1(\mathbf{z}_0)))) \triangleq \mathbf{f}_\theta(\mathbf{z}_0)$$

- Can transform simple distributions into a more complex one:
 - Base distribution: Gaussian



- Base distribution: Uniform



Normalizing flow models

- Learning via **maximum likelihood** over the dataset $\mathcal{D}$

$$\max_{\theta} \log p_X(\mathcal{D}; \theta) = \sum_{\mathbf{x} \in \mathcal{D}} \log p_Z(\mathbf{f}_{\theta}^{-1}(\mathbf{x})) + \log \left| \det \left(\frac{\partial \mathbf{f}_{\theta}^{-1}(\mathbf{x})}{\partial \mathbf{x}} \right) \right|$$

Normalizing flow models

- Learning via **maximum likelihood** over the dataset $\mathcal{D}$

$$\max_{\theta} \log p_X(\mathcal{D}; \theta) = \sum_{\mathbf{x} \in \mathcal{D}} \log p_Z(\mathbf{f}_{\theta}^{-1}(\mathbf{x})) + \log \left| \det \left(\frac{\partial \mathbf{f}_{\theta}^{-1}(\mathbf{x})}{\partial \mathbf{x}} \right) \right|$$

- **Exact likelihood evaluation** via inverse transformation $\mathbf{x} \rightarrow \mathbf{z}$ and change of variables formula
- **Sampling** via forward transformation $\mathbf{z} \rightarrow \mathbf{x}$

$$\mathbf{z} \sim p_Z(\mathbf{z}) \quad \mathbf{x} = \mathbf{f}_{\theta}(\mathbf{z})$$

Normalizing flow models

- Learning via **maximum likelihood** over the dataset $\mathcal{D}$

$$\max_{\theta} \log p_X(\mathcal{D}; \theta) = \sum_{\mathbf{x} \in \mathcal{D}} \log p_Z(\mathbf{f}_{\theta}^{-1}(\mathbf{x})) + \log \left| \det \left(\frac{\partial \mathbf{f}_{\theta}^{-1}(\mathbf{x})}{\partial \mathbf{x}} \right) \right|$$

- **Exact likelihood evaluation** via inverse transformation $\mathbf{x} \rightarrow \mathbf{z}$ and change of variables formula
- **Sampling** via forward transformation $\mathbf{z} \rightarrow \mathbf{x}$

$$\mathbf{z} \sim p_Z(\mathbf{z}) \quad \mathbf{x} = \mathbf{f}_{\theta}(\mathbf{z})$$

- **Latent representations** inferred via inverse transformation

$$\mathbf{z} = \mathbf{f}_{\theta}^{-1}(\mathbf{x})$$

Key points for normalizing flows

- **Simple prior** $p_z(z)$ that allows for efficient sampling and tractable likelihood evaluation.

Key points for normalizing flows

- **Simple prior** $p_z(z)$ that allows for efficient sampling and tractable likelihood evaluation.
- **Invertible transformations with tractable evaluation:**
 - Likelihood evaluation requires efficient evaluation of $x \rightarrow z$ mapping
 - Sampling requires efficient evaluation of $z \rightarrow x$ mapping

Key points for normalizing flows

- **Simple prior** $p_z(z)$ that allows for efficient sampling and tractable likelihood evaluation.
- **Invertible transformations with tractable evaluation:**
 - Likelihood evaluation requires efficient evaluation of $x \rightarrow z$ mapping
 - Sampling requires efficient evaluation of $z \rightarrow x$ mapping
- Computing likelihoods also require **the evaluation of determinants of $n \times n$ Jacobian matrices**, where n is the data dimensionality
 - Computing the determinant for an $n \times n$ matrix is **$O(n^3)$** , which is expensive
- **Transformations with special structured Jacobian matrix.** For example, the determinant of a triangular matrix is the product of the diagonal entries, i.e., an $O(n)$ operation

Attempts of finding invertible transformation

- **NICE (Nonlinear Independent Components Estimation)** (Dinh et al, 2014)
 - Triangular weight matrices, $M = LU$

Attempts of finding invertible transformation

- **NICE (Nonlinear Independent Components Estimation) (Dinh et al, 2014)**
 - Triangular weight matrices, $M = LU$
 - General coupling layer

$$\begin{aligned}y_{I_1} &= x_{I_1} \\ y_{I_2} &= g(x_{I_2}; m(x_{I_1}))\end{aligned}$$

where g is an invertible map.

Jacobian

$$\frac{\partial y}{\partial x} = \begin{bmatrix} I_d & 0 \\ \frac{\partial y_{I_2}}{\partial x_{I_1}} & \frac{\partial y_{I_2}}{\partial x_{I_2}} \end{bmatrix}$$

Invert mapping as

$$\begin{aligned}x_{I_1} &= y_{I_1} \\ x_{I_2} &= g^{-1}(y_{I_2}; m(y_{I_1}))\end{aligned}$$

Attempts of finding invertible transformation

- **NICE (Nonlinear Independent Components Estimation) (Dinh et al, 2014)**
 - Triangular weight matrices, $M = LU$
 - General coupling layer
 - Additive coupling layers

Partition the variables $\mathbf{z}$ into two disjoint subsets, say $\mathbf{z}_{1:d}$ and $\mathbf{z}_{d+1:n}$ for any $1 \leq d < n$

- Forward mapping $\mathbf{z} \mapsto \mathbf{x}$:
 - $\mathbf{x}_{1:d} = \mathbf{z}_{1:d}$ (identity transformation)
 - $\mathbf{x}_{d+1:n} = \mathbf{z}_{d+1:n} + m_{\theta}(\mathbf{z}_{1:d})$ ($m_{\theta}(\cdot)$ is a neural network with parameters θ , d input units, and $n - d$ output units)
- Inverse mapping $\mathbf{x} \mapsto \mathbf{z}$:
 - $\mathbf{z}_{1:d} = \mathbf{x}_{1:d}$ (identity transformation)
 - $\mathbf{z}_{d+1:n} = \mathbf{x}_{d+1:n} - m_{\theta}(\mathbf{x}_{1:d})$
- Jacobian of forward mapping:

$$J = \frac{\partial \mathbf{x}}{\partial \mathbf{z}} = \begin{pmatrix} I_d & 0 \\ \frac{\partial \mathbf{x}_{d+1:n}}{\partial \mathbf{z}_{1:d}} & I_{n-d} \end{pmatrix}$$

$$\det(J) = 1$$

- **Volume preserving transformation** since determinant is 1.

Attempts of finding invertible transformation

- **NICE (Nonlinear Independent Components Estimation) (Dinh et al, 2014)**
 - Triangular weight matrices, $M = LU$
 - General coupling layer
 - Additive coupling layers
 - Rescaling layers
 - Additive coupling layers are composed together (with arbitrary partitions of variables in each layer)
 - Final layer of NICE applies a rescaling transformation
 - Forward mapping $\mathbf{z} \mapsto \mathbf{x}$:

$$x_i = s_i z_i$$

where $s_i > 0$ is the scaling factor for the i -th dimension.

- Inverse mapping $\mathbf{x} \mapsto \mathbf{z}$:

$$z_i = \frac{x_i}{s_i}$$

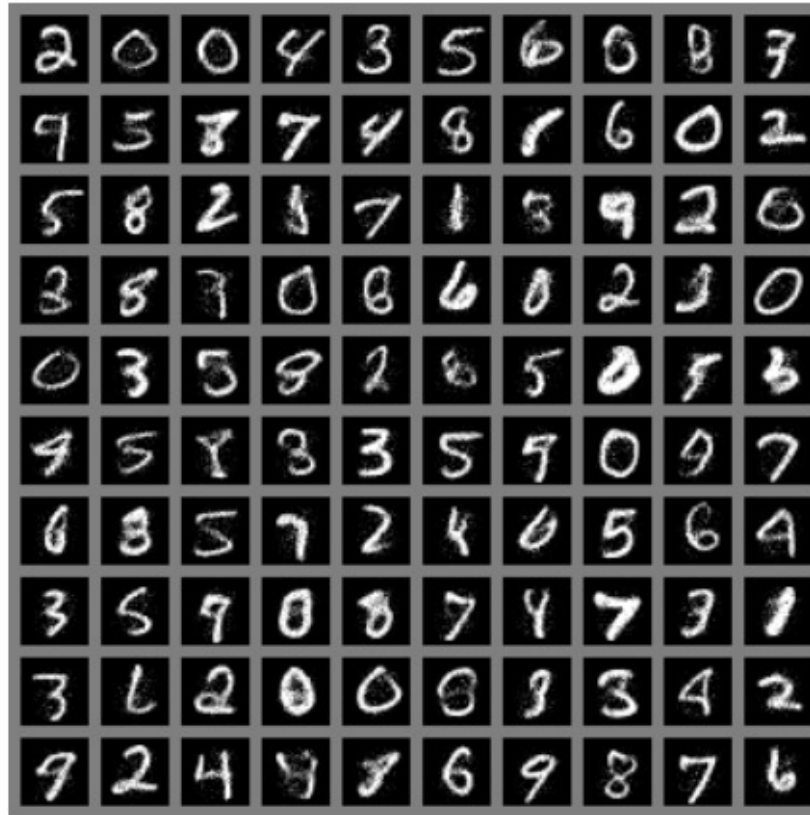
- Jacobian of forward mapping:

$$J = \text{diag}(\mathbf{s})$$

$$\det(J) = \prod_{i=1}^n s_i$$

Attempts of finding invertible transformation

- NICE (Nonlinear Independent Components Estimation) (Dinh et al, 2014)



(a) Model trained on MNIST



(b) Model trained on TFD

Attempts of finding invertible transformation

- **Real-NVP (Non-volume Preserving Extension of NICE) (Dinh et al, 2017)**
 - Forward mapping $\mathbf{z} \mapsto \mathbf{x}$:
 - $\mathbf{x}_{1:d} = \mathbf{z}_{1:d}$ (identity transformation)
 - $\mathbf{x}_{d+1:n} = \mathbf{z}_{d+1:n} \odot \exp(\alpha_\theta(\mathbf{z}_{1:d})) + \mu_\theta(\mathbf{z}_{1:d})$
 - $\mu_\theta(\cdot)$ and $\alpha_\theta(\cdot)$ are both neural networks with parameters θ , d input units, and $n - d$ output units [$\odot$ denotes elementwise product]
 - Inverse mapping $\mathbf{x} \mapsto \mathbf{z}$:
 - $\mathbf{z}_{1:d} = \mathbf{x}_{1:d}$ (identity transformation)
 - $\mathbf{z}_{d+1:n} = (\mathbf{x}_{d+1:n} - \mu_\theta(\mathbf{x}_{1:d})) \odot (\exp(-\alpha_\theta(\mathbf{x}_{1:d})))$
 - Jacobian of forward mapping:

$$J = \frac{\partial \mathbf{x}}{\partial \mathbf{z}} = \begin{pmatrix} I_d & 0 \\ \frac{\partial \mathbf{x}_{d+1:n}}{\partial \mathbf{z}_{1:d}} & \text{diag}(\exp(\alpha_\theta(\mathbf{z}_{1:d}))) \end{pmatrix}$$

$$\det(J) = \prod_{i=d+1}^n \exp(\alpha_\theta(\mathbf{z}_{1:d})_i) = \exp\left(\sum_{i=d+1}^n \alpha_\theta(\mathbf{z}_{1:d})_i\right)$$

- **Non-volume preserving transformation** in general since determinant can be less than or greater than 1

Attempts of finding invertible transformation

- **Real-NVP (Non-volume Preserving Extension of NICE) (Dinh et al, 2017)**
 - Forward mapping $\mathbf{z} \mapsto \mathbf{x}$:
 - $\mathbf{x}_{1:d} = \mathbf{z}_{1:d}$ (identity transformation)
 - $\mathbf{x}_{d+1:n} = \mathbf{z}_{d+1:n} \odot \exp(\alpha_\theta(\mathbf{z}_{1:d})) + \mu_\theta(\mathbf{z}_{1:d})$
 - $\mu_\theta(\cdot)$ and $\alpha_\theta(\cdot)$ are both neural networks with parameters θ , d input units, and $n - d$ output units [$\odot$ denotes elementwise product]
 - Inverse mapping $\mathbf{x} \mapsto \mathbf{z}$:
 - $\mathbf{z}_{1:d} = \mathbf{x}_{1:d}$ (identity transformation)
 - $\mathbf{z}_{d+1:n} = (\mathbf{x}_{d+1:n} - \mu_\theta(\mathbf{x}_{1:d})) \odot (\exp(-\alpha_\theta(\mathbf{x}_{1:d})))$
 - Jacobian of forward mapping:

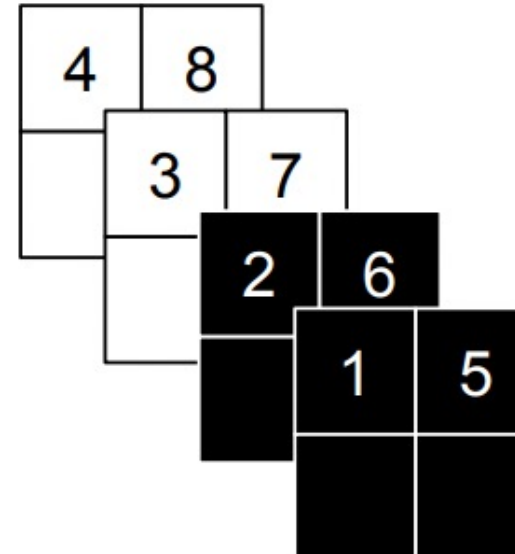
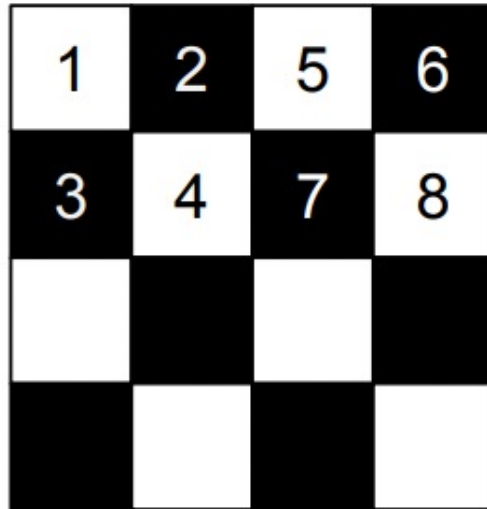
$$J = \frac{\partial \mathbf{x}}{\partial \mathbf{z}} = \begin{pmatrix} I_d & 0 \\ \frac{\partial \mathbf{x}_{d+1:n}}{\partial \mathbf{z}_{1:d}} & \text{diag}(\exp(\alpha_\theta(\mathbf{z}_{1:d}))) \end{pmatrix}$$

$$\det(J) = \prod_{i=d+1}^n \exp(\alpha_\theta(\mathbf{z}_{1:d})_i) = \exp\left(\sum_{i=d+1}^n \alpha_\theta(\mathbf{z}_{1:d})_i\right)$$

- **Non-volume preserving transformation** in general since determinant can be less than or greater than 1

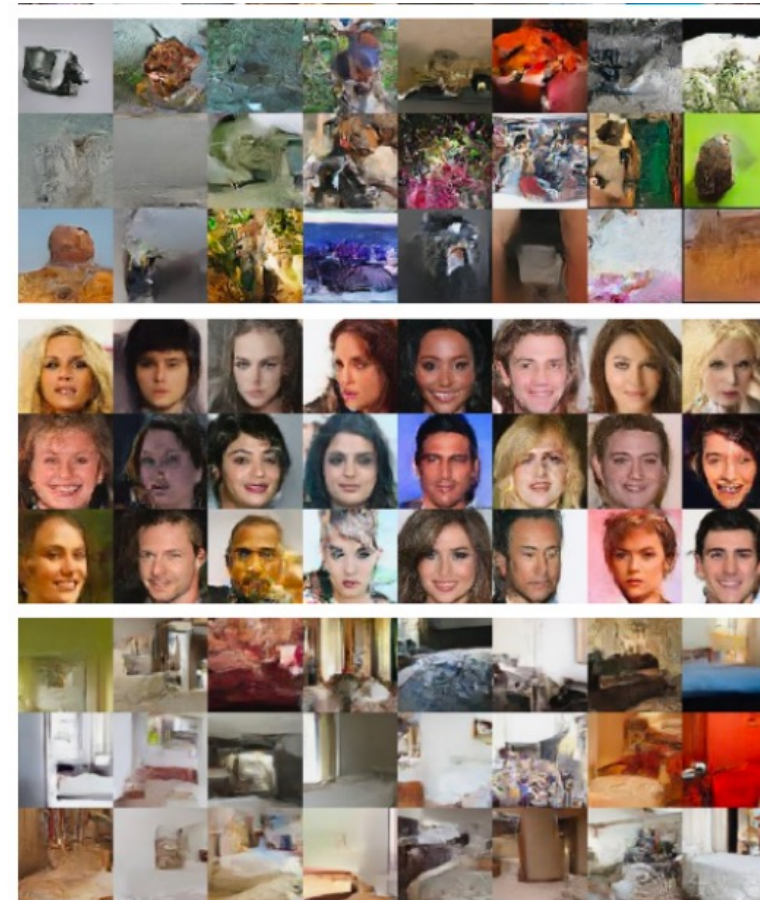
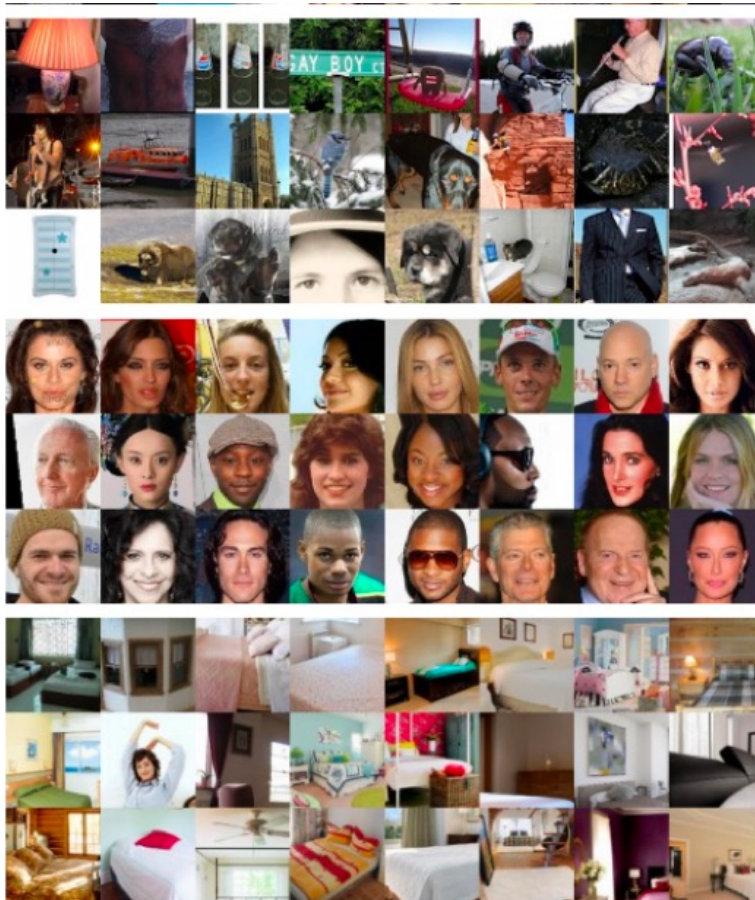
Attempts of finding invertible transformation

- **Real-NVP (Non-volume Preserving Extension of NICE) (Dinh et al, 2017)**
 - Forward mapping $\mathbf{z} \mapsto \mathbf{x}$:
 - $\mathbf{x}_{1:d} = \mathbf{z}_{1:d}$ (identity transformation)
 - $\mathbf{x}_{d+1:n} = \mathbf{z}_{d+1:n} \odot \exp(\alpha_\theta(\mathbf{z}_{1:d})) + \mu_\theta(\mathbf{z}_{1:d})$
 - $\mu_\theta(\cdot)$ and $\alpha_\theta(\cdot)$ are both neural networks with parameters θ , d input units, and $n - d$ output units [$\odot$ denotes elementwise product]



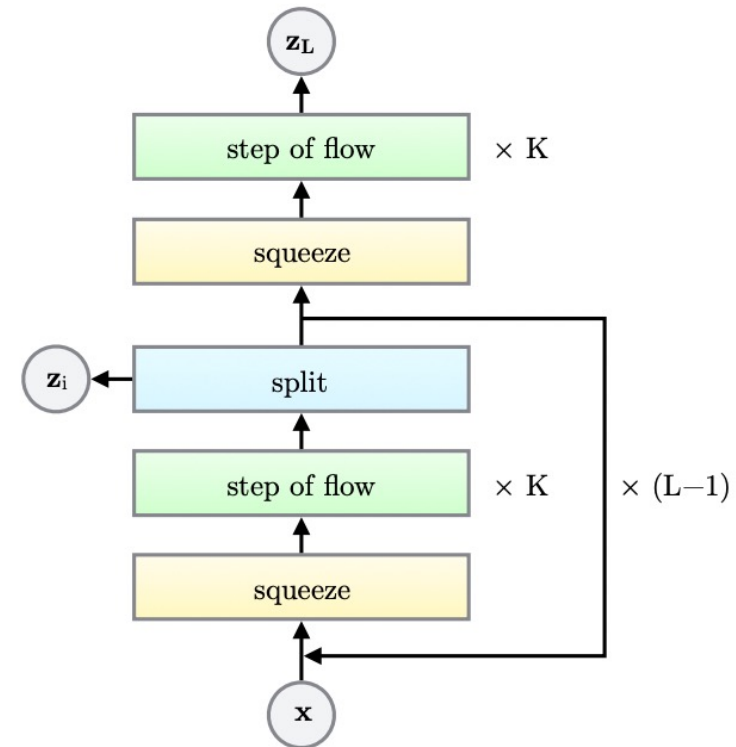
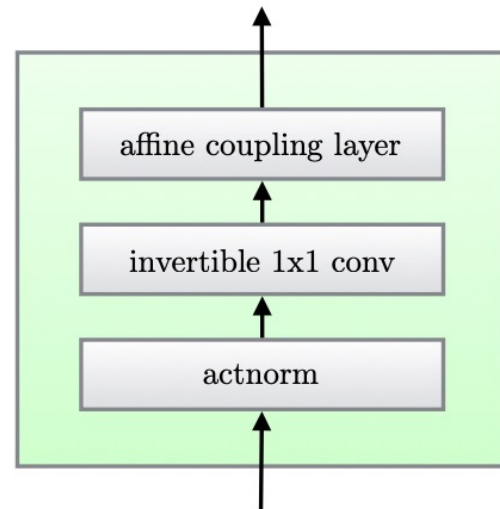
Attempts of finding invertible transformation

- Real-NVP (Non-volume Preserving Extension of NICE) (Dinh et al, 2017)



Attempts of finding invertible transformation

- **Glow: Generative Flow with Invertible 1x1 Convolutions (Kingma et al, 2018)**



Attempts of finding invertible transformation

- **Glow: Generative Flow with Invertible 1x1 Convolutions (Kingma et al, 2018)**

LU Decomposition. This cost of computing $\det(\mathbf{W})$ can be reduced from $\mathcal{O}(c^3)$ to $\mathcal{O}(c)$ by parameterizing $\mathbf{W}$ directly in its LU decomposition:

$$\mathbf{W} = \mathbf{P}\mathbf{L}(\mathbf{U} + \text{diag}(\mathbf{s})) \quad (10)$$

where $\mathbf{P}$ is a permutation matrix, $\mathbf{L}$ is a lower triangular matrix with ones on the diagonal, $\mathbf{U}$ is an upper triangular matrix with zeros on the diagonal, and $\mathbf{s}$ is a vector. The log-determinant is then simply:

$$\log |\det(\mathbf{W})| = \text{sum}(\log |\mathbf{s}|) \quad (11)$$

Attempts of finding invertible transformation

- **Glow: Generative Flow with Invertible 1x1 Convolutions (Kingma et al, 2018)**



Attempts of finding invertible transformation

- Autoregressive models as flow models
 - Consider a Gaussian autoregressive model:

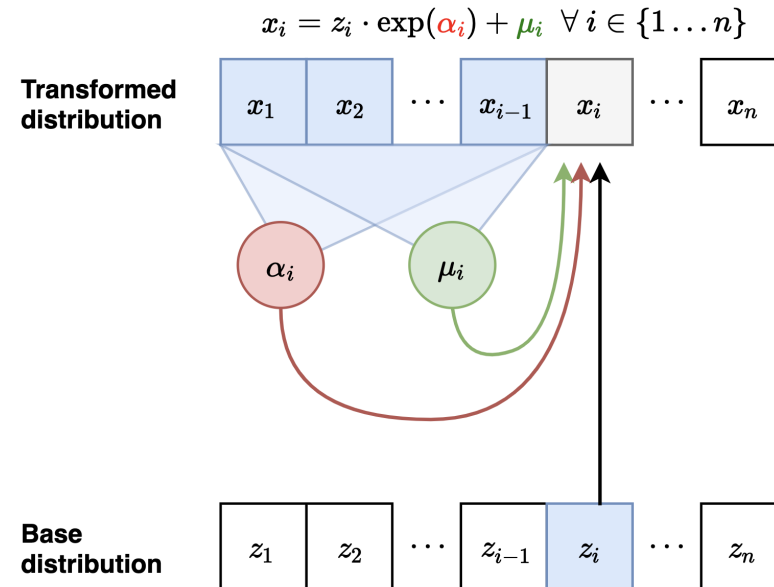
$$p(\mathbf{x}) = \prod_{i=1}^n p(x_i | \mathbf{x}_{<i})$$

such that $p(x_i | \mathbf{x}_{<i}) = \mathcal{N}(\mu_i(x_1, \dots, x_{i-1}), \exp(\alpha_i(x_1, \dots, x_{i-1})))^2$.
Here, $\mu_i(\cdot)$ and $\alpha_i(\cdot)$ are neural networks for $i > 1$ and constants for $i = 1$.

- Sampler for this model:
 - Sample $z_i \sim \mathcal{N}(0, 1)$ for $i = 1, \dots, n$
 - Let $x_1 = \exp(\alpha_1)z_1 + \mu_1$. Compute $\mu_2(x_1), \alpha_2(x_1)$
 - Let $x_2 = \exp(\alpha_2)z_2 + \mu_2$. Compute $\mu_3(x_1, x_2), \alpha_3(x_1, x_2)$
 - Let $x_3 = \exp(\alpha_3)z_3 + \mu_3$
- **Flow interpretation:** transforms samples from the standard Gaussian $(z_1, z_2, \dots, z_n)$ to those generated from the model $(x_1, x_2, \dots, x_n)$ via invertible transformations (parameterized by $\mu_i(\cdot), \alpha_i(\cdot)$)

Attempts of finding invertible transformation

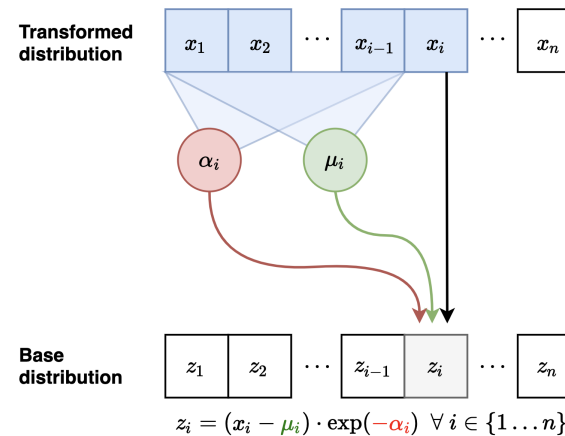
- **MAF: Masked Autoregressive Flow** (Papamakarios et al. 2017)



- Forward mapping from $\mathbf{z} \mapsto \mathbf{x}$:
 - Let $x_1 = \exp(\alpha_1)z_1 + \mu_1$. Compute $\mu_2(x_1), \alpha_2(x_1)$
 - Let $x_2 = \exp(\alpha_2)z_2 + \mu_2$. Compute $\mu_3(x_1, x_2), \alpha_3(x_1, x_2)$
- Sampling is sequential and slow (like autoregressive): $O(n)$ time

Attempts of finding invertible transformation

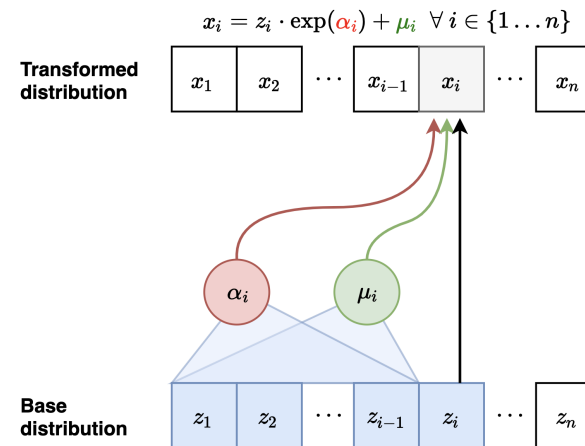
- **MAF: Masked Autoregressive Flow (Papamakarios et al. 2017)**



- Inverse mapping from $\mathbf{x} \mapsto \mathbf{z}$:
 - Compute **all** μ_i, α_i (can be done in parallel)
 - Let $z_1 = (x_1 - \mu_1) / \exp(\alpha_1)$ (scale and shift)
 - Let $z_2 = (x_2 - \mu_2) / \exp(\alpha_2)$
 - Let $z_3 = (x_3 - \mu_3) / \exp(\alpha_3) \dots$
- Jacobian is lower diagonal, hence efficient determinant computation
- Likelihood evaluation is easy and parallelizable
- Layers with different variable orderings can be stacked

Attempts of finding invertible transformation

- IAF: Inverse Autoregressive Flow (Kingma et al. 2016)



- Forward mapping from $\mathbf{z} \mapsto \mathbf{x}$ (parallel):
 - Sample $z_i \sim \mathcal{N}(0, 1)$ for $i = 1, \dots, n$
 - Compute all μ_i, α_i (can be done in parallel)
 - Let $x_1 = \exp(\alpha_1)z_1 + \mu_1$
 - Let $x_2 = \exp(\alpha_2)z_2 + \mu_2 \dots$
- Inverse mapping from $\mathbf{x} \mapsto \mathbf{z}$ (sequential):
 - Let $z_1 = (x_1 - \mu_1) / \exp(\alpha_1)$. Compute $\mu_2(z_1), \alpha_2(z_1)$
 - Let $z_2 = (x_2 - \mu_2) / \exp(\alpha_2)$. Compute $\mu_3(z_1, z_2), \alpha_3(z_1, z_2)$
- Fast to sample from, slow to evaluate likelihoods of data points (train)
- Note: Fast to evaluate likelihoods of a generated point (cache $z_1, z_2, \dots, z_n$)

Attempts of finding invertible transformation

- **IAF: Inverse Autoregressive Flow (Kingma et al. 2016)**

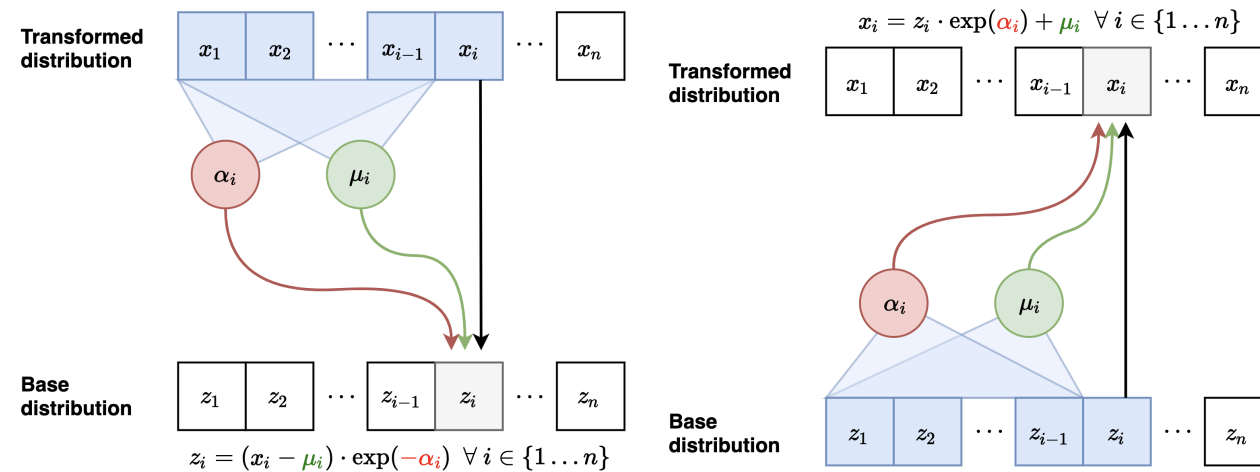


Figure: Inverse pass of MAF (**left**) vs. Forward pass of IAF (**right**)

- Interchanging **z** and **x** in the inverse transformation of MAF gives the forward transformation of IAF
- Similarly, forward transformation of MAF is inverse transformation of IAF

Attempts of finding invertible transformation

- **IAF: Inverse Autoregressive Flow (Kingma et al. 2016)**
 - Computational tradeoffs
 - MAF: Fast likelihood evaluation, slow sampling
 - IAF: Fast sampling, slow likelihood evaluation
 - MAF more suited for training based on MLE, density estimation
 - IAF more suited for real-time generation
 - Can we get the best of both worlds?
 - Two part training with a teacher and student model
 - Teacher is parameterized by MAF. Teacher can be efficiently trained via MLE
 - Once teacher is trained, initialize a student model parameterized by IAF. Student model cannot efficiently evaluate density for external datapoints but allows for efficient sampling

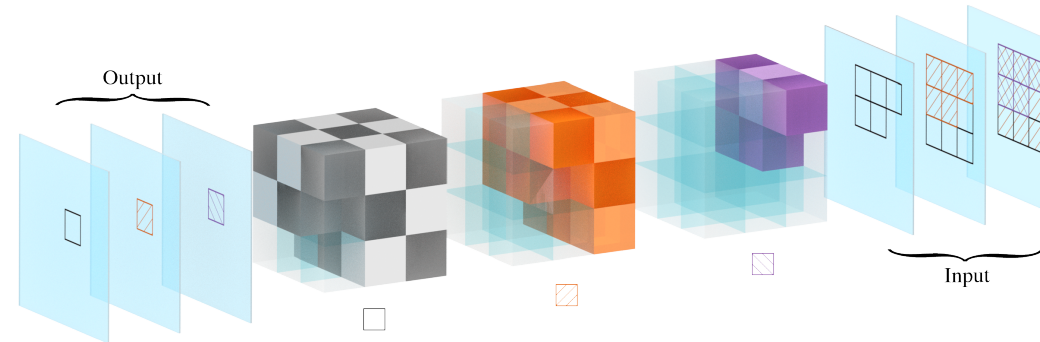
Attempts of finding invertible transformation

- **MintNet (Song et al. 2019)**
 - MintNet: Building invertible neural networks with masked convolutions.
 - A regular convolutional neural network is powerful, but it is not invertible and its Jacobian determinant is expensive.
 - We can instead use masked convolutions like in autoregressive models to enforce ordering (like PixelCNN)
 - Because of the ordering, the Jacobian matrix is triangular and the determinant is efficient to compute.
 - If all the diagonal elements of the Jacobian matrix are (strictly) positive, the transformation is invertible.

Attempts of finding invertible transformation

- **MintNet (Song et al. 2019)**

- Illustration of a masked convolution with 3 filters and kernel size 3×3 .



- Solid checkerboard cubes inside each filter represent unmasked weights, while the transparent blue blocks represent the weights that have been masked out.
- The receptive field of each filter on the input feature maps is indicated by regions shaded with the pattern (the colored square) below the corresponding filter.

Attempts of finding invertible transformation

- Many more...
- And **flow matching**



To be continued